Finally, we took this to the next level and deployed our app to the Cloud, using a three-node cluster on both DigitalOcean and AWS, and we used this to demonstrate Swarm's scaling capabilities.

That's a heck of a lot to have achieved (especially for a book of this size!). I'll leave the choice of celebration in your capable hands, although I'd urge you to think bigger than the proverbial pat on the back.

However, although we've started our journey toward production, there's too much to cover to get to a fully resilient, CI/CD pipeline-driven, secure, scalable production environment. Hopefully, you've had a taste of the power that Docker offers, and you can see its potential and are inspired to learn more.

This short chapter is a grab bag of various things we couldn't squeeze in elsewhere. As well as planting some seeds of ideas in your mind that you'll find useful, this chapter will provide you some pointers on continuing your journey with Docker.

What Should I Learn About Next?

Great question. As you no doubt are aware, our learning never ends. The more you know, the more you realize you don't know. C'est la vie.

To help in this effort, I'm going to share my thoughts on (what I consider to be) valuable areas for further learning. For each topic, I'll provide a brief summary, but you'll have to do a bit of digging to learn more and think about how to apply these to your application. Some areas will be more or less appealing and more or less useful in your situation, so dig deeper into the areas you need, and feel free to leave what you don't.

Enjoy!

Limiting Resources

As you run more containers in your cluster, you may find the need to constrain the CPU resources and memory given to certain containers. Both Swarm[1] and Kubernetes[2] provide a way to specify limits on the resources a container is allowed.

Here's an example with Swarm that means containers for `some-service` will only receive a maximum of 50 MB of memory, and one tenth of a single CPU core.

1. https://docs.docker.com/config/containers/resource_constraints/
2. https://kubernetes.io/docs/concepts/configuration/manage-compute-resources-container/